

ReproBot: A Multi-Agent System for Automated Scientific Paper Replication

Third Progress Report

Aleyna Kütük
aleynakutuk10@gmail.com

Engincan Varan
engincanvaran@gmail.com

Gül Korkut
serifegulkorkut@gmail.com

Göktuğ Mert Özdoğan
goekmeroz@gmail.com

inzva AI Projects #10
13.09.2026

Abstract

The second progress report left ReproBot with a working five-stage pipeline and no replication result: full-fidelity training on the CIFAR-10 targets was measured at roughly 22 days per paper on the available CPU, and no reproduced number had been compared against any claim. This report covers the phase that changed both. We redirected the evaluation towards papers whose original experiments train in minutes on a CPU, and, at the same time, generalized the code-generation and execution stages beyond image classification: the Coder now writes a plain PyTorch training loop for any supervised task, under a task-agnostic metrics contract. Two new papers were carried through every stage at full fidelity, producing the project’s first replication measurements, compared by hand. The reproduction of Tang’s *Deep Learning using Linear Support Vector Machines* reaches 0.82% MNIST test error against a claimed 0.87% — but only after one hyperparameter the paper never states was corrected by hand, following an ablation that diagnosed why the first full run collapsed into a network that ignored its input. The reproduction of Wijaya’s housing-price model reaches a test RMSE of 4.48 against a claimed 3.02, a gap we cannot attribute from a single seed because the paper does not specify its data split. Along the way the retry loop repaired its first naturally occurring defect, and six prompt failures observed in real runs were each converted into an explicit rule. We also report the team’s first front end, a Streamlit dashboard over the pipeline’s outputs. The central lesson of the phase is that every stage reported *success* on a run that had completely failed: nothing in ReproBot yet judges whether a number is plausible, which makes the Critic the most important remaining component.

1 Introduction

The first progress report [1] set out ReproBot’s motivation and design, and the second [2] documented its implementation: PDF extraction, a five-field structured Reader, a Coder, a Docker-sandboxed Runner, and an Orchestrator running a Coder $\leftrightarrow$ Runner retry loop. That report closed on two limitations stated without qualification. No replication-fidelity claim was possible, since the Critic did not exist and nothing compared a reproduced number with a paper’s. And no fidelity claim was even *measurable*, since the only full training run the hardware could support would have taken weeks.

This phase addressed the second limitation directly and the first by hand. Its three threads are reported in turn. Section 2 describes a change of direction: evaluating on papers small enough to train for real on a CPU, and a team decision that ReproBot must reproduce more

Table 1: The second report’s stated limitations, revisited.

Stated on 22.08.2026	Status on 13.09.2026
No fidelity claim is possible yet	Two claims measured at full fidelity and compared by hand (Section 5.4); the Critic still does not exist
Full-fidelity runs infeasible (~22 days per CIFAR-10 paper)	Unchanged for CIFAR-10; sidestepped with two papers whose full runs took 32 and 54 minutes on the same CPU
The evidence base is two papers	Four papers carried end to end: two beyond CIFAR-10, one of them tabular regression
The retry loop was demonstrated only on an injected fault	It repaired a naturally occurring defect (Section 5.2)
Four of seven verdicts exercised only against test doubles	<code>environment_error</code> occurred in a real run — wrongly assigned, and the triage prompt corrected (Section 4); three remain unobserved
Retries regenerate rather than patch	Unchanged, and now measured: a one-argument fix produced a script only 22% similar to its predecessor
Validation does not converge; two papers cannot be extracted	Unchanged

than one kind of paper. Section 3 describes the resulting changes to the Coder and Runner, and Section 4 the prompt failures that real runs exposed and the rules that now prevent them. Section 5 reports the measurements themselves, including a training collapse, the ablation that explained it, and the first two comparisons against published claims. Section 6 describes the team’s pipeline viewer. Sections 7–12 collect the engineering findings, the methodology, progress against the original timeline, limitations, and next steps.

Since the second report, the repository has gained seven commits, which together added 933 lines and removed 280 in the pipeline stages, and a feature branch has added a 544-line dashboard. Table 1 states, for each limitation the second report recorded, where it now stands.

2 A Change of Direction

2.1 The compute wall

The CIFAR-10 papers chosen as ReproBot’s first targets are the natural benchmark for the proposal’s image-classification scope, and they are unreachable on the hardware available to the project. At the measured throughput of 5.3 training samples per second, one *Wide Residual Networks* run at the paper’s own setup takes approximately 22 days. The second report’s reproduced numbers were therefore all drawn from stages that train for one epoch on a few hundred images, and could say nothing about fidelity.

Securing GPU compute remains necessary for that benchmark. It is not, however, necessary to test the claim ReproBot actually makes — that a paper can be read, implemented, executed and checked automatically — which requires only a paper whose original experiment is small. We therefore looked for papers whose headline result trains in minutes on a CPU, while leaving the curated CIFAR-10 set untouched.

2.2 More than one kind of paper

The search surfaced a structural problem. The pipeline was, in practice, hardwired to image classification: the Coder’s prompt assumed `torchvision` datasets and the HuggingFace `Trainer` image-model interface, the metrics contract was keyed on accuracy, and the Runner’s image carried no tabular libraries. An in-scope MNIST paper would have tested the pipeline as it stood. A housing-price regression paper could not have run at all.

The team’s decision was that this was a defect rather than a scope boundary: a system that replicates only one task type is not a general replication system, and ReproBot should handle several. CIFAR-10 remains the main evaluation set, but no stage may now assume images. Both candidate papers were retained, precisely because they differ:

- **Tang (2013)**, *Deep Learning using Linear Support Vector Machines* [3]. A classification paper whose contribution is a loss: the softmax output layer is replaced by a linear L2-SVM trained with a squared hinge loss. The targeted claim is 0.87% MNIST test error. The setup — PCA to 70 dimensions, two 512-unit hidden layers, 400 epochs — is stated in unusual detail, which makes the two values it omits all the more consequential (Section 5.3).
- **Wijaya (2023)**, *Multi Level Dense Layer Neural Network Model for Housing Price Prediction* [4]. A tabular regression paper in Keras, on the Boston Housing data [5]: 506 rows, 13 features, a stated 405/101 train/test split, 1000 epochs. The targeted claim is a test RMSE of 3.02. It tests three things CIFAR-10 cannot: a non-image data source, a regression metric where lower is better, and a paper written for a different framework.

Both were added under a separate directory, so the curated CIFAR-10 set remains exactly as its curator left it.

3 Generalizing the Coder and Runner

3.1 Dropping the HuggingFace Trainer

The Coder previously emitted scripts built on HuggingFace’s **Trainer**. Its model contract — image tensors in, logits out — fits a custom margin loss or a regression head poorly, and that alone argued for replacing it. The deciding reason, however, was that **Trainer** applies defaults that no paper states, and this was demonstrated rather than assumed.

When the generalized Coder regenerated its *Network In Network* script as a regression test, the new plain-PyTorch script kept the paper’s architecture — a genuine `mlpconv` cascade with no fully-connected layer — and passed its execution probe. But its loss reached approximately 947, where the previous **Trainer**-based script had reported 2.30. The cause was a default: the old script never set `max_grad_norm`, and therefore inherited **Trainer**’s gradient clipping at a norm of 1.0, which we confirmed inside the container. The paper uses no clipping. Faithful to the paper and without its unstated initialization, the same learning rate diverges.

We record this as a correction to the second report. Its *Network In Network* runs executed under a stabilizing mechanism the paper does not describe; their numbers were already labelled as non-replication results, but the scripts were less faithful than that report implied. The Coder is now forbidden from using **Trainer**, **accelerate**, **Lightning** or **Keras**, and writes an explicit training loop in which every mechanism is visible.

3.2 Always PyTorch, whatever the paper used

Wijaya’s model was built in Keras. Rather than generate code for a second framework, the Coder always writes PyTorch, and the prompt treats the source framework’s defaults as part of the method: where a paper is silent, it inherited its framework’s behaviour, so a faithful translation must reproduce that behaviour. The prompt names the Keras defaults that differ — Adam’s $\epsilon = 10^{-7}$, Glorot initialization, batch normalization with momentum 0.99 and $\epsilon = 10^{-3}$, and a `fit()` batch size of 32 — and requires every remaining difference to be disclosed. Section 7 reports that this rule was followed only in part.

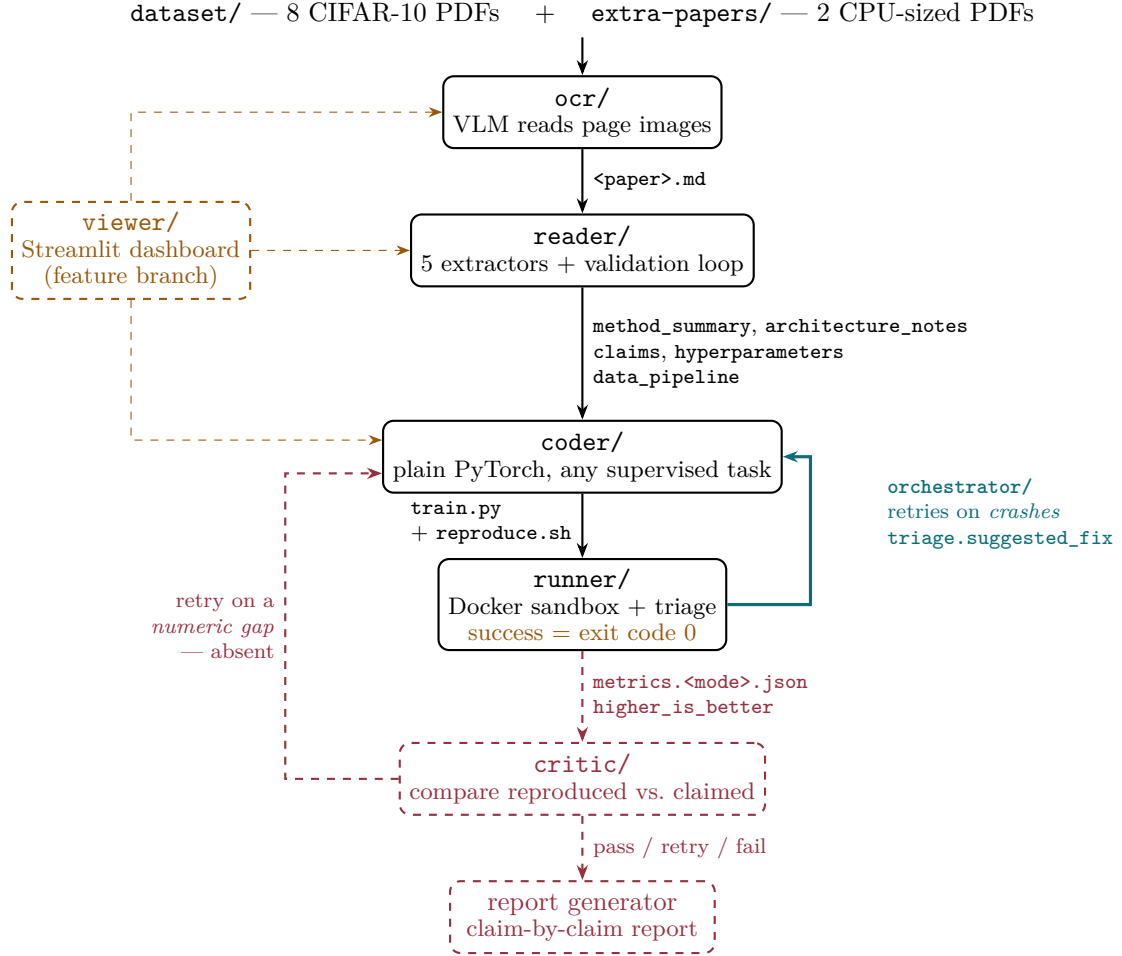


Figure 1: ReproBot on 13.09.2026. Solid components are built and verified by execution; dashed red components do not exist; the dashed amber component is on a feature branch. The pipeline’s shape is unchanged from the second report, but its inputs now include two CPU-sized papers, one of them tabular regression, and the Coder writes a plain PyTorch loop for any supervised task. The label inside the Runner marks this phase’s central finding: a run is reported as a success when its script exits normally, whatever number it produced.

3.3 A task-agnostic metrics contract

Every generated script writes a fixed-shape metrics file that the Runner parses and a future Critic will read. The second report’s contract assumed accuracy. Its generalization is summarized in Table 2; the most consequential addition is **higher_is_better**, since a bare value of 4.0 is meaningless without knowing whether it is an error or an accuracy.

3.4 Data sourcing, and verifying what was fetched

Image datasets still come from `torchvision`. Tabular data is fetched from OpenML, pinned by dataset identifier, before any direct URL is considered, and every download lands in the shared dataset cache the Runner mounts. The preference order was not a design choice made in advance; Section 4 describes the failure that produced it.

The generated script must also check, in code and before training, that the dataset it fetched is the one it intended — its name, row count and column count — and raise otherwise. The motivation is that a wrong identifier does not fail. It downloads a different dataset that happens to carry that number, trains on it, and reports a plausible metric for the wrong problem. Where

Table 2: Changes to the metrics contract. `metric` and `unit` are copied verbatim from the targeted claim, so a comparison against the claim needs no conversion. Files in the old shape still parse.

Field	Change
<code>train_metric, eval_metric</code>	Replace <code>train_accuracy</code> and <code>eval_accuracy</code> : the claim’s own metric on each split
<code>higher_is_better</code>	New; the direction of the metric
<code>task_type</code>	New; inferred by the Coder from the claim, e.g. <code>classification</code> or <code>regression</code>
<code>value, metric, unit</code>	Unchanged; the headline number and its claim-verbatim label

a paper gives split sizes but not how the split was drawn, the script uses a seeded random split and records the assumption.

3.5 Runner changes

The Runner’s interface did not change: it still invokes only the generated `reproduce.sh` and one of its four modes, which is what allowed two new task types to be added without modifying its code. Its image gained pinned `pandas`, `scikit-learn` and `scipy`, installed in the same `pip` command as the pinned `numpy`, so that a transitive upgrade fails the build rather than silently moving a numerical dependency.

4 Prompt Hardening from Real Failures

Running the generalized pipeline on real papers exposed six failures in the prompts of the Coder and of the Runner’s triage step. None was found by inspection; each surfaced in a run. Table 3 lists them with the rule that now addresses each. Every triage rule was verified by re-running triage on the real log that exposed the problem.

Two of these generalize beyond this project, and are drawn out in Section 7. Rows 1 and 5 share a pattern: in both, a model supplied an identifier from memory that looked exactly as plausible as a correct one. Row 6 is the converse, and the more uncomfortable: the defect was in our own prompt, whose descriptive wording the model implemented literally.

5 Results

5.1 Extraction on the new papers

The Reader required no changes for either paper. Table 4 extends the second report’s coverage table with both. Each targeted claim was extracted exactly, as was Tang’s distinction between the L1- and L2-SVM gradients.

Three weaknesses surfaced, all recorded as open defects. First, validation still does not converge: both new papers end with five unresolved flags. Second, the same result is extracted two or three times when a paper states it in prose, a table and a figure; Wijaya’s 14 claims describe eight distinct results, at mixed precision (0.911 and 0.91). A Critic comparing against claims will need them deduplicated. Third, and most consequential, the Reader records what a paper leaves unstated only for its *architecture*. Tang’s paper states “stochastic gradient descent with momentum” without a momentum value, and an L2-SVM objective without its penalty constant C . Neither omission was flagged anywhere — and they are exactly the two values that collapsed training (Section 5.3).

Table 3: Prompt failures observed in real runs, in the order they occurred, and the rule that now addresses each. Rows 1–5 arose from the first tabular run; row 6 from its rerun.

	Observed failure	Consequence	Rule now in the prompt
1	The Coder fetched Boston Housing from a remembered CMU StatLib URL, which now returns HTTP 403 to every client	The first execution stage failed in 5 s	Coder: OpenML by identifier first; a direct URL only for data found nowhere else
2	Triage classified the 403 as <code>environment_error</code>	The Orchestrator stopped with zero retries on a failure one regeneration would fix	Triage: a refusal from one source is recoverable; only a container with no network at all is environmental
3	A re-triage suggested switching to “a standard alternative regression dataset”	The fix would change what is being reproduced	Never change what is being reproduced
4	A re-triage suggested <code>load_boston</code>	That loader was removed from scikit-learn in version 1.2	Never suggest an API or source not certainly current; give one fix, not a menu
5	A re-triage named Boston Housing as OpenML <code>data_id=506</code>	Identifier 506 is a different dataset (Boston Housing is 531); training would silently proceed on it	Name no identifier absent from the log; the Coder asserts the dataset’s name and shape in code
6	The Coder’s prompt twice said “OpenML <code>data_id</code> and version”; the script passed both	<code>fetch_openml</code> rejects that combination with a <code>ValueError</code>	Pin by identifier alone, or by name together with a version

Table 4: Reader output across six papers; the first four rows are reproduced from the second report. *Comp.* is architecture components; *own/eq* is equations marked as the paper’s own over equations captured; *Gaps* is `unstated_details` entries; *Flags* is unresolved validation flags after the three-pass cap.

Paper	Claims	Hyp.	Data	Comp.	own/eq	Gaps	Flags
Network In Network	12	17	4	7	1/3	12	4
All Convolutional Net	17	13	4	10	2/6	11	5
Deep Residual Learning	66	28	6	10	2/3	7	8
Wide Residual Networks	61	20	4	11	1/1	11	6
Tang (2013), DLSVM	15	23	3	10	4/11	12	5
Wijaya (2023), housing	14	10	1	8	2/2	8	5

5.2 The retry loop repairs a real defect

The second report demonstrated the retry loop only against a deliberately reintroduced fault. Wijaya’s two orchestrated runs, shown in Figure 2, supply the first naturally occurring case — and, in the row before it, the failure that motivated the triage rules of Section 4.

Before the prompts were hardened, the first execution stage failed on the dead URL, triage called it environmental, and the loop stopped as designed for an environmental fault — correctly given its input, and wrongly in fact. After hardening, the regenerated script fetched the right dataset but passed both an identifier and a version to `fetch_openml`, which raised. Triage classified the failure as recoverable and proposed removing the `version` argument, which matches the traceback exactly. The Coder regenerated with that feedback, the plateau guard measured a similarity of 0.22, and the second attempt asserted the dataset’s name and shape

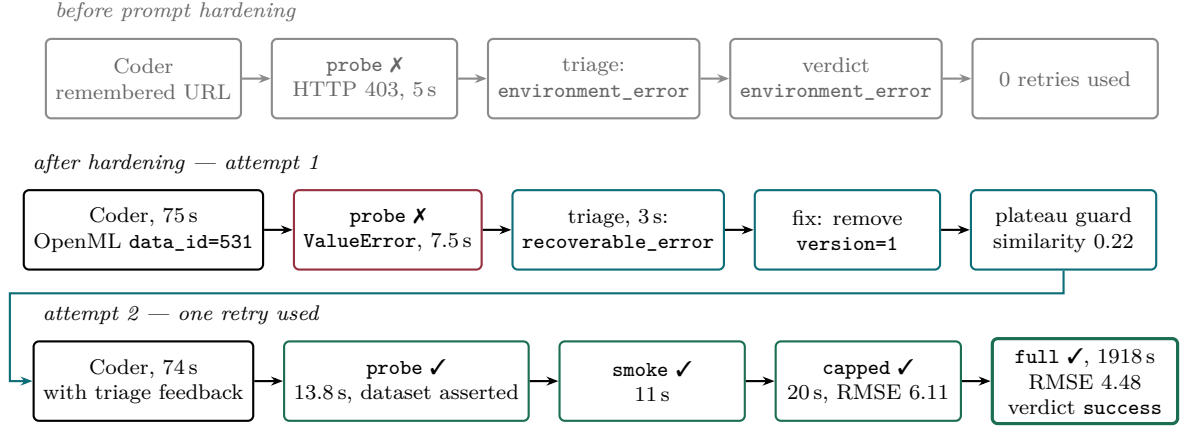


Figure 2: Wijaya’s orchestrated runs. Top: before the prompt changes of Section 4, a dead URL was classified as an environmental fault and the loop stopped without retrying. Middle and bottom: after them, a genuine defect in the regenerated script was triaged correctly, the Coder regenerated with the one-line fix as feedback, and the retry passed every stage. The plateau guard’s 0.22 is far below its 0.98 stopping threshold — and shows how much of the script changed to remove one argument.

Table 5: The collapsed run against a network that ignores its input. The small excess is the L2 penalty and the input noise.

	Input-ignoring optimum	Observed
Training loss	3.599	~3.607
Training error	88.8%	~89.5%

and passed every stage through **full**. The whole repair cost two generation calls, one triage call, and under three minutes.

5.3 The first full run, and its collapse

Tang’s generated script passed **probe**, **smoke** and **capped** with training error falling from 75% to 31% — evidence, as far as those stages can provide it, that the network learned. Its **full** run was ReproBot’s first at a paper’s own scale. By epoch 20 its training error had risen to 83%, and it then settled near 89.5% (Figure 3). It was noticed by a team member reading the live training log. No stage noticed, and had the run completed, the Runner would have reported *success*: the script exited normally and wrote a well-formed metrics file.

Diagnosis. For a one-vs-rest squared hinge loss, a network that ignores its input can do no better than a constant score per class, $s_k = 2p_k - 1$, where p_k is the frequency of class k . The resulting loss is $\sum_k 4p_k(1 - p_k)$, and the best such network always predicts the most frequent digit. Evaluated at MNIST’s class frequencies, both quantities match the observed run (Table 5). The network had died: early oversized updates drove every hidden unit’s pre-activation negative, so every ReLU output zero for every input and no gradient reached the hidden layers; only the output bias still learned, to exactly the input-ignoring optimum. A decaying learning rate cannot recover from this, since a zero gradient stays zero at any step size.

Ablation. The Coder had disclosed four values the paper does not state: momentum 0.9, $C = 1.0$, PyTorch’s default initialization, and unwhitened PCA features. To isolate the cause, we wrote a harness that imports the model, loss and data loading *from the generated script itself*, holds every paper-stated value fixed, and varies one of the four guesses at a time for 30

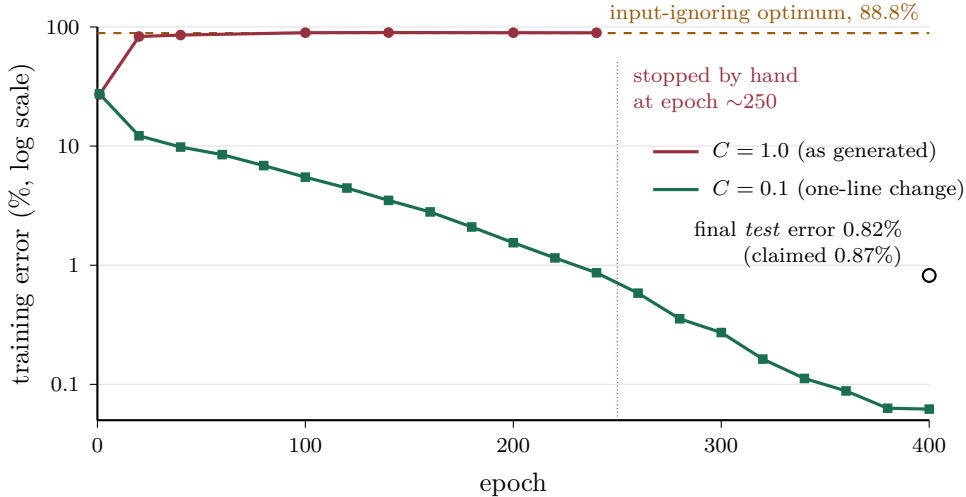


Figure 3: Training error for Tang’s reproduction at the paper’s full setting, from the run logs. As generated ($C = 1.0$, momentum 0.9), the network learned for a few epochs and then collapsed onto the error of a network that ignores its input; the run was stopped by hand. With the single change $C = 0.1$, training error fell smoothly to 0.062%, and the test set, evaluated once after the final epoch, gave 0.82% against the claimed 0.87%. The loss values of the two runs are not comparable, since C scales the loss, so error is plotted.

Table 6: Ablation over the four unstated values, each changed alone from the generated configuration (A). *Dead* is the fraction of second-layer hidden units that output zero for all 2,000 probe images at epoch 30; first-layer units were never dead in any configuration.

	Configuration	Test @5	Test @30	Dead @30	Outcome
A	As generated (mom. 0.9, $C = 1.0$)	6.6%	47.1%	0.71	unstable
B	Momentum 0.5	4.0%	2.62%	0.38	stable
C	Momentum 0.0	4.2%	2.66%	0.14	stable
D	$C = 0.1$	4.4%	2.80%	0.04	stable, healthiest
E	PCA whitened	79.8%	90.3%	0.996	total collapse
F	Init. $\mathcal{N}(0, 0.01)$	5.0%	4.2%	0.31	stable, slower

epochs of the paper’s schedule, measuring the fraction of hidden units that output zero for every one of 2,000 probe images. Table 6 reports the six configurations.

The collapse requires momentum 0.9 and $C = 1.0$ *together*: reducing either alone stabilizes training. Momentum 0.9 amplifies the effective step roughly tenfold, and $C = 1.0$ scales the squared-hinge gradient tenfold relative to $C = 0.1$. Each value is a reasonable default in isolation; the pair is too aggressive for this loss at the paper’s learning rate of 0.1. Whitening the PCA features enlarges the inputs — the mean squared norm rises from 46 to 70 — and collapses fastest, which supports the Coder’s literal, unwhitened reading. Configuration A degraded but did not fully collapse within 30 epochs while the real run did; the two used different random streams for the input noise, so the collapse is seed-dependent, which is precisely what makes it hard to catch with short runs.

We selected $C = 0.1$ (configuration D) as a one-line change to the existing script rather than a regeneration, so that nothing else could change at the same time. It keeps the momentum consistent with the paper’s “with momentum”, produced the healthiest network, and concerns a constant the paper itself treats as tuned elsewhere.

Table 7: ReproBot’s first full-fidelity measurements, compared against the claims by hand. One seed each. Runtimes are wall clock on the development CPU, with the two runs sharing it.

Paper	Claim	Claimed	Reproduced	Runtime	Reading
Tang (2013)	MNIST test error	0.87%	0.82%	3,223 s	Consistent with the claim, after one unstated value was corrected by hand
Wijaya (2023)	Test RMSE	3.02	4.48	1,918 s	Not reproduced on this split
Wijaya (2023)	Train RMSE	2.69	2.94	—	Close; see text

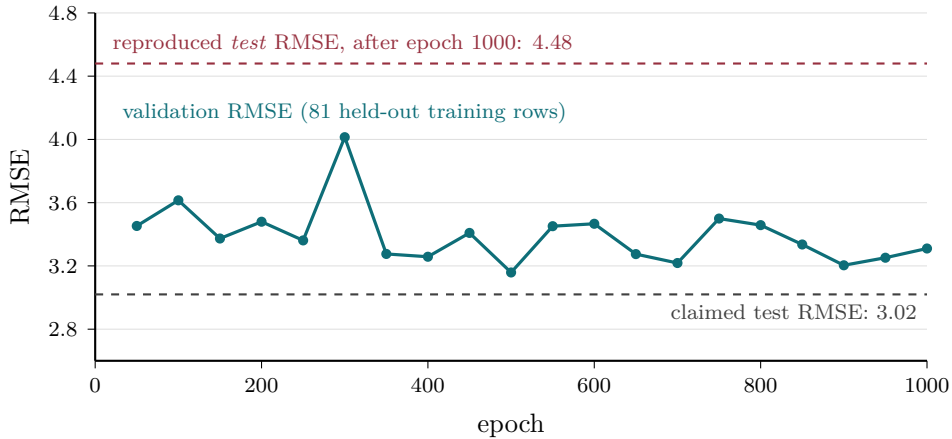


Figure 4: Wijaya’s reproduction at the paper’s full setting, from the run log. The model trained on 324 rows; RMSE on 81 further held-out training rows (every 50 epochs; epoch 1, at 20.7, is off the scale) stayed near the claimed test RMSE throughout, while the 101-row test set, evaluated once at the end, gave 4.48. Two held-out subsets of the same 506-row dataset disagree by more than the gap being measured, which is why a single split cannot adjudicate this reproduction.

5.4 First fidelity measurements

Both papers then completed **full** runs at their papers’ stated settings. Table 7 reports the results against the published claims.

Tang. With $C = 0.1$, training error fell smoothly to 0.062% with no sign of collapse (Figure 3), and the test set, evaluated once after the final epoch, gave 0.82%. The difference from the claim is five test images out of 10,000. With a single seed we cannot measure run-to-run variance; as a reference scale, the binomial standard error of a 0.85% error rate over 10,000 samples is about 0.09 percentage points. We therefore read the result as consistent with the claim, not as an improvement on it.

Two qualifications matter for how this result may be cited. It is not autonomous: a human chose C . And although the ablation selected on network health — dead units and distance from the input-ignoring loss — it also logged short-horizon test error, so the choice was not made blind to the test set. The accurate statement is that *the generated implementation replicates the paper’s claim once one hyperparameter the paper omits is corrected*.

Wijaya. The reproduced test RMSE of 4.48 is 48% above the claim, while the training RMSE of 2.94 is within 9% of it. The training log qualifies the headline number (Figure 4). The generated script, following the paper’s description, held out 20% of the training rows for monitoring. On those 81 rows, which the model never trained on, the RMSE stayed between 3.2 and 3.5 for most of training and ended at 3.31 — close to the claimed test value. Only the

101-row test set produced 4.48. Two held-out subsets of the same 506-row dataset thus disagree by more than the gap being measured.

The paper states its split sizes but not how rows were assigned, so the script used a seeded random split. On a dataset this small, with a target censored at its maximum value, which rows fall into a 101-row test set plausibly dominates the result. A single seed cannot separate an unfaithful implementation from an unlucky split, and we do not claim either. Two further candidates remain untested: the Coder did not reproduce Keras’s batch-normalization and initialization defaults (Section 7), and a constant learning rate left the final epoch’s training loss noisy.

5.5 Cost

One Coder call consumed approximately 27,000 input and 9,500 output tokens and took 75 s; one triage call, about 2,400 input and 240 output tokens, in 3 s. Compute again dominated, and in a way worth recording. Both full runs shared the CPU, and each container claimed every core. Wijaya’s 1000 epochs over 324 training rows took 1,906 s — about 1.9 s per epoch of 11 small batches, which is thread contention rather than model cost — and Tang’s run took 54 minutes. The Runner deliberately leaves CPU limits unset, because an arbitrary limit produces failures indistinguishable from crashes; the practical consequence is that concurrent runs should be avoided.

6 A Front End: The Pipeline Viewer

In parallel with the pipeline work, the team built ReproBot’s first user interface: **viewer/**, a dashboard written in Streamlit, developed on a feature branch. The original proposal planned a Gradio demonstration for the final month; the viewer is an early step towards it, and is already useful as a working tool.

What it shows. The main page lists every paper with a status table — whether OCR, Reader and Coder output exist, and how many validation flags remain. Selecting a paper opens tabs for its method summary, architecture notes, claims, hyperparameters, data pipeline, validation flags, raw extraction JSON, generated training script with its **reproduce.sh**, and the raw OCR Markdown. A reviewer can therefore inspect what the Reader extracted and what the Coder did with it side by side, without opening files.

What it can run. An “Import a paper” section accepts a PDF upload, with an optional page limit so that a new paper can be tested cheaply before committing to the whole document. Per-paper buttons then run Reader extraction or code generation. Two design decisions make this safe. Every action calls the corresponding stage’s own entry point, unmodified, so the viewer cannot drift from the command-line pipeline. And uploaded papers are stored in a separate directory rather than the curated dataset directory, which is maintained by another team member.

Status. The branch was verified with strict type checking, the repository’s pre-commit hooks, and a headless Streamlit run that rendered every tab against synthetic fixtures. It is not yet merged. Two gaps remain before it is: it does not yet display Runner or Orchestrator output — the results this report is about — and it predates the generalization of the Coder and Runner, so its dependency changes need reconciling with the updated lock file before it can merge. Extending it to show a run’s attempts, triage decisions and metrics, and eventually the Critic’s verdict, is the natural route to the planned demonstration.

7 Engineering Findings

7.1 Execution success is not evidence of a result

The second report argued that static checks cannot establish that generated code runs. This phase established the next step: that code running to completion cannot establish that it worked. Tang’s collapsed run would have exited normally and been reported as a success. The escalation ladder could not have caught it — `capped` trains for about fifteen steps, and the collapse needed thousands — and `capped`’s stated purpose, checking that training learns, is not enforced by anything: the Runner reads only the exit code.

The same pattern appeared at smaller scale throughout the phase. The `full` mode of every generated `reproduce.sh` wrote its metrics to a different filename from the one the Runner reads, so both fidelity results above reached the Runner only through its fallback parser; this is now fixed. The OCR, Reader and Coder stages exit with status zero even when every paper fails, which is how a missing image-library dependency went unnoticed on the first OCR attempt of the phase. Every such seam is a place where failure can look like success.

7.2 Framework defaults are part of the method

Two findings point the same way. HuggingFace `Trainer`’s default gradient clipping made a script look stable that, written faithfully, diverges. And a paper written in Keras inherits Keras’s defaults wherever it is silent, so reproducing it faithfully in another framework means reproducing those defaults. The Coder was instructed to do so, and followed the instruction only partly: Wijaya’s script matched Keras’s Adam ϵ but used PyTorch’s batch-normalization momentum and ϵ and PyTorch’s default initialization, and disclosed the choice rather than following the rule. Disclosure makes a deviation visible; it does not make it faithful.

7.3 A remembered identifier is worse than a missing one

A model that lacks an identifier fails visibly. A model that recalls one wrongly — a dead URL, a removed loader, a dataset number one digit away from correct — produces an artifact that looks correct and, for a dataset identifier, trains without complaint on the wrong data. The defence that worked was not asking the model to be more careful, but removing the opportunity: triage may not name identifiers absent from the log, and the generated script must verify what it fetched before using it.

7.4 Prompt wording is executed literally

The one defect this phase introduced into generated code came from our own prompt, which described OpenML datasets as pinned by “data_id and version”. The description was meant loosely; the model passed both arguments. Prompts for code generators are read as specifications, and descriptive phrasing in them needs the same care as an API contract.

7.5 Unstated values fail jointly

The Coder’s disclosure discipline worked as designed: momentum 0.9 and $C = 1.0$ were both listed as guesses. What it could not express was that the *combination* was unsafe. Reporting each unstated value in isolation is necessary but not sufficient; the values that matter most may be the ones that interact, and finding that out required an ablation. This argues for recording unstated hyperparameters as carefully as unstated architecture, and for a cheap automated sensitivity probe over them.

Table 8: Status against the first report’s staged timeline. ✓ done; ∼ partial or in a different form; ✗ not started.

Phase	Planned	Status
Month 1	PDF ingestion; Reader schema; shared-memory object	✓ Complete; five fields
Month 2	Coder and Docker sandbox for a pilot set; Orchestrator skeleton with a single hard-coded retry	✓ Exceeded: a bounded retry loop with triage-driven feedback
Month 3	Critic verdict logic	✗ Not started; first real inputs now exist
	Real retry loop with targeted feedback	✓ Repaired a real defect
	Report Generator	✗ Not started
	Single-shot vs. iterative ablation	∼ One observed case only: Wijaya fails single-shot and passes on the first retry
Month 4	Expansion towards 20 papers	∼ Four papers; classification and regression
	Demonstration interface	∼ Streamlit viewer on a feature branch

7.6 One seed cannot adjudicate a gap

Wijaya’s two held-out subsets disagreed by more than the gap between the reproduction and the claim. A Critic that compares one run against one published number under a fixed tolerance would declare this reproduction a failure — and would declare a lucky split a success. A tolerance must account for the variance of the quantity being compared, which in practice means several seeds per claim.

7.7 Regeneration rewrites, measurably

The second report noted that retries regenerate whole scripts rather than patching them. Wijaya’s repair quantifies the cost: triage’s fix was the removal of one keyword argument, and the regenerated script shared only 22% of its lines with its predecessor. The repair succeeded, but almost everything else in the script also changed, unreviewed. The plateau guard’s threshold of 0.98 now has two observations — 0.41 on the second report’s injected fault and 0.22 on this real one — and neither came near it.

8 Development Methodology

The phase continued the delegation pattern described in the second report, though most of its work was investigative rather than constructive and was carried out directly by the orchestrating session. One scoped implementation — the generalization of the Coder and Runner — was delegated to a coding subagent.

That delegation produced the third instance, across the project, of a subagent’s report being wrong in a way only independent verification caught. The agent reported its *Network In Network* regression test as a clean pass. It had passed: the script executed and the metrics file was well formed. The loss was approximately 947. The discrepancy was found on review, and led directly to the hidden-default finding above. The practice of treating an agent’s report as a claim to check against the diff and the output remains the most useful single habit we have adopted.

The two most consequential decisions of the phase were human: the direction change to support multiple task types, and the choice of C for Tang’s rerun. The collapse itself was

spotted by a person reading a log. We state this plainly because an automated replication system is exactly the kind of project in which human intervention can be quietly absorbed into a result.

9 Progress Against the Timeline

Table 8 compares the phase’s output with the staged timeline of the first report. The retry loop arrived ahead of plan and in stronger form than planned. The Critic and the Report Generator, both Month 3 deliverables, have not started, and the phase’s effort went instead to generalization and to obtaining the first measurable results — a trade we consider correct, since a Critic without measurable results would have had nothing to judge.

10 Limitations

Both fidelity results are single runs, compared by hand. Neither has a variance estimate, and no stage of the system performed either comparison.

The positive result required a human. Tang’s reproduction matches its claim only after a person chose one hyperparameter, informed by an ablation that had logged test error.

The negative result is not attributed. Wijaya’s gap may stem from the unstated split, from framework defaults the Coder did not reproduce, or from the implementation, and one seed cannot distinguish them.

The CIFAR-10 benchmark remains unmeasured. GPU compute is still required for it, and the two CPU-sized papers are not substitutes for the planned evaluation, only a way to test the pipeline’s claim end to end.

The Reader does not record missing hyperparameters, and claims are duplicated across a paper’s prose, tables and figures. Validation still does not converge, and two of the eight CIFAR-10 papers still cannot be extracted.

A note on the dataset. Boston Housing includes a feature derived from the proportion of Black residents in each town [5], which is among the reasons scikit-learn deprecated and then removed its loader. We used it only because the paper under replication did, and do not recommend it for any other purpose.

11 Next Steps

Build the Critic, with variance in mind. Its verdict should remain explicit arithmetic against a numeric tolerance using the claim’s direction, as planned; Wijaya shows that the tolerance must be informed by several seeds per claim rather than one run. Tang’s collapsed run is its first test case: a verdict of *fail* at 89% error against 0.87% is the minimum it must deliver.

Make the Runner check that training learns. Reading `train_metric` at capped, and comparing a run’s loss against the trivial input-ignoring floor, would have flagged Tang’s collapse within twenty epochs. This is smaller than the Critic and catches a different failure.

Record and probe unstated hyperparameters. Extend the Reader’s gap-recording to hyperparameters, and automate what the Tang ablation did by hand: short runs over candidate values for the unstated ones, selected on training health and a validation split — never the test set.

Patch rather than regenerate. A line-range edit, as in AutoReproduce [6], would keep a one-argument fix to one argument.

Merge and extend the viewer to show Runner, Orchestrator and Critic output, as the basis of the final demonstration.

Secure GPU compute for the CIFAR-10 set, and build the Report Generator and a single entry point from PDF to report, after fixing the stages that exit successfully on failure.

12 Conclusion

This phase produced ReproBot’s first replication measurements. One reproduction matches its published claim to within five test images out of ten thousand; the other misses its claim by a margin that a single run cannot explain. Getting there required generalizing the pipeline beyond image classification, converting six real prompt failures into explicit rules, and diagnosing a training collapse that every stage of the pipeline reported as a success.

That last point is the phase’s main result. ReproBot can now read a paper, write an implementation, execute it at the paper’s own scale, and repair its own execution failures. What it cannot yet do is tell whether the number it produced is right. The two measurements in this report were judged by people, and one of them needed a person to fix it first. Building the component that makes those judgements automatically, honestly and with appropriate uncertainty, is the work of the next phase.

References

- [1] Kütük, A., Varan, E., Korkut, G., & Özdoğan, G. M. (2026). ReproBot: A Multi-Agent System for Automated Scientific Paper Replication — First Progress Report. *inzva AI Projects #10*.
- [2] Kütük, A., Varan, E., Korkut, G., & Özdoğan, G. M. (2026). ReproBot: A Multi-Agent System for Automated Scientific Paper Replication — Second Progress Report. *inzva AI Projects #10*.
- [3] Tang, Y. (2013). Deep Learning using Linear Support Vector Machines. *ICML 2013 Workshop on Challenges in Representation Learning*. arXiv:1306.0239.
- [4] Wijaya, R. (2023). Multi Level Dense Layer Neural Network Model for Housing Price Prediction. *arXiv:2310.08133*.
- [5] Harrison, D., & Rubinfeld, D. L. (1978). Hedonic housing prices and the demand for clean air. *Journal of Environmental Economics and Management*, 5(1), 81–102.
- [6] Zhao, X., Sun, M., Wang, W., et al. (2025). AutoReproduce: Automatic AI Experiment Reproduction with Paper Lineage. *arXiv:2505.20662*.